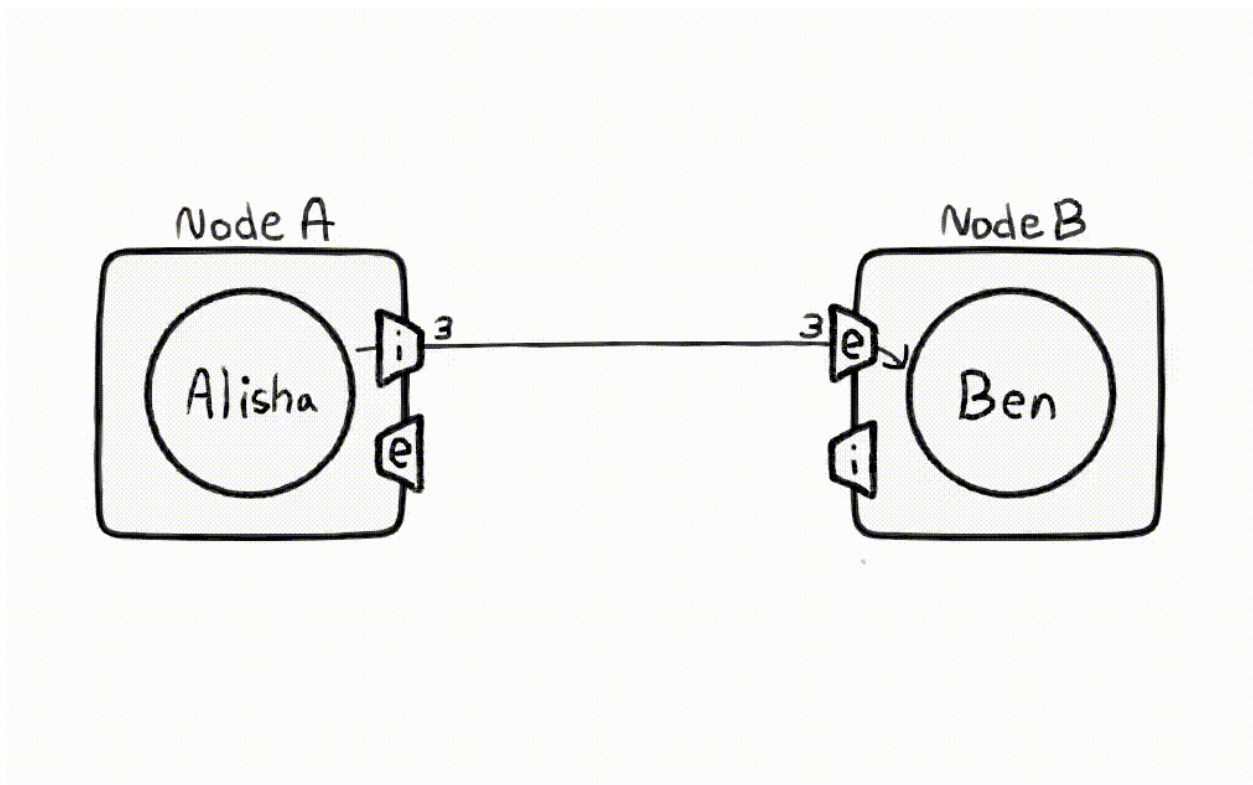


Introducing OCapN, interoperable capabilities over the network — Spritely Institute

Introducing OCapN, interoperable capabilities over the network

Jessica Tallon — November 16, 2023



OCapN (**O**bject **C**apability **N**etwork) is a new protocol to add support for secure, seamless, and distributed (often peer-to-peer) communication for networked applications.

OCapN was originally developed as part of Goblins, our distributed object programming environment. But OCapN is no longer just for Goblins! Thanks to the support of a grant from NLnet, we have documented OCapN's functionality as a set of specifications so that OCapN's technology can be incorporated into any programming language environment. We have been working with colleagues across several partner projects and organizations to shape these documents following a consensus-based review process.

Without OCapN, building networked applications such as social networks, chat rooms and online virtual worlds otherwise would have to manually solve many problems which OCapN handles automatically as part of its design. OCapN simplifies building otherwise complicated peer-to-peer systems and has many useful features:

- **Intuitive distributed programming:** allows for networked programming such that it doesn't matter where objects live: they can be fully local or distributed over the network. Performing asynchronous operations against networked objects is the same as asynchronous programming against local ones.
- **Object capability security:** makes complicated secure systems simpler to reason about following the principle of "if you don't have it, you can't use it."
- **Promise Pipelining:** enables sending messages to a promise's potentially settled output. This streamlines reasoning about promises, and cuts down on round trips across the network.
- **Distributed Garbage Collection:** powers garbage collection across multiple devices. While programming languages may sufficiently handle garbage collection within a local machine, when references to objects are scattered across a network, the network itself must also explicitly support garbage collection of obsolete objects.
- **Certificate-Based Third-Party Handoffs:** provide a mechanism by which two machines can securely reference objects on a third. Connections between machines that potentially span across networks necessitate the free sharing of object references, even those which exist on machines outside that connection. Third-party handoffs enable such references to be intuitively and securely shared within Goblins, without needing to consider the network.

NLnet Grant

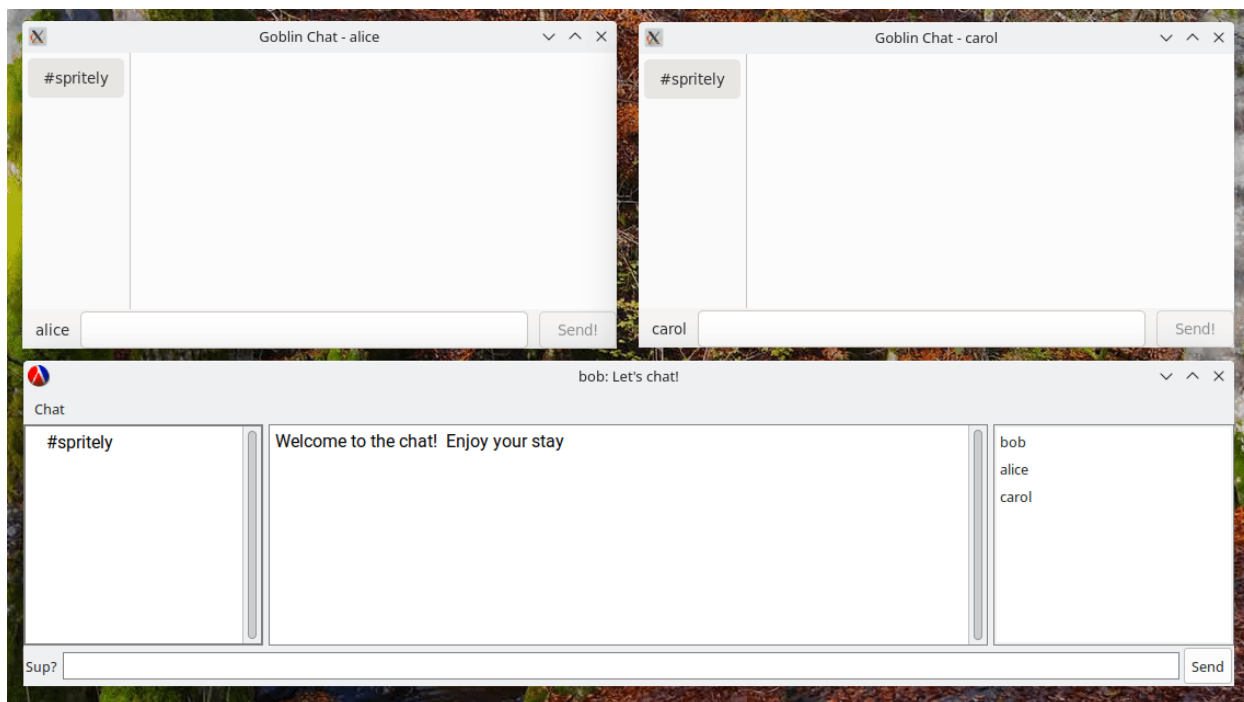
Jessica Tallon has been working with Spritely on an NLnet grant-funded initiative for the past year, with a focus on improving OCapN support in both Goblins implementations and initiating the standardization push for OCapN. Below, Jessica delivers her personal account of her efforts to develop, grow, and refine open standards for the protocol to ensure solid foundations and interoperability across the OCapN ecosystem over the course of the grant.

Milestone 1: Racket Goblins <-> Guile Goblins Interoperability

When I started on this grant work in late 2022, Racket Goblins had a mature OCapN implementation, but Guile Goblins had none. This meant the two implementations couldn't interoperate (send messages to each other) at all.

At the start of 2023, I completed this milestone, adding comprehensive OCapN support to Guile Goblins and a protocol to enable streamlined communication with its Racket counterpart.

To assess the boundaries of OCapN coverage, the team and I ported a simple and minimalist chat application used internally for testing from Racket Goblins to Guile Goblins.



The evaluation was successful, meaning that Goblins objects written in Guile scheme can send messages with those written in Racket scheme and vice versa over OCapN.

Milestone 2: Start the OCapN Pre-standardization Group

Although we've made significant additions to the OCapN standard — such as certificate-based third-party handoffs — through our work on its Goblins implementation, it's important to note that the protocol wasn't invented at Spritely; we've drawn from and built upon the contributions of many others (the E programming language, Agoric, and Cap'n Proto to name a few) and is actively being shaped by input from many developers of those projects.

The OCapN group is comprised of many organizations such as Spritely and Agoric as well as many independent experts and interested parties. The group's goal is to come to consensus on draft documents which define the protocols within OCapN.

If you're interested in joining the group and helping the standardization effort, or you just want to take a look at our work, please find more info at ocapn.org

Milestone 3: Three draft specifications

Part of the standardization effort is to have documents the OCapN group can develop as the group meets consensus. The initial draft specifications are based on the Goblins' implementations of OCapN, The OCapN protocol is split into three documents:

- CapTP: covers the majority of the protocol including message sending, promises, promise pipelining, third-party handoffs.
- Netlayers: describes how new netlayer types should be defined and documents the ones currently in use, such as the Tor Onion Services Netlayer. Netlayers are an abstraction for different transport protocols which CapTP uses to communicate on, this is because OCapN is designed to be agnostic to the specific protocol CapTP runs on. While Goblins so far has largely used Tor, as it provides strong anonymity, other protocols, such as libp2p, IBC or carrier pigeons with little flash drives could be used.
- Locators: defines how representations of nodes (machines) on the network and specific objects are encoded. The node identity specifications include both in-band and out-of-band, string-like represen-

tations, both of which are needed in order to bootstrap new connections and to share references within established connections.

As the OCapN group progresses these standardization efforts, we'll continue to update these documents; and on Spritely's end, our Goblins OCapN implementation.

Milestone 4: Test suite

One of the main goals of the OCapN standardization effort is to develop an open standard with many implementations. To ensure all of them interoperate as expected, the OCapN test suite verifies that a given implementation has correctly implemented the protocol according to spec.

Excitingly, this test suite has already been put to use: Alexander Bondarenko's Haskell implementation of OCapN has been built against and passes the OCapN test suite!

Milestone 5: Both goblins implementations pass test suite

Perhaps it goes without saying, but Spritely's Racket and Guile Goblins' OCapN implementations should also both pass the test suite. This may seem simple because the specification documents, and therefore the test suite itself, were written based on our Goblins' OCapN implementations. However the standard drafts continued to change, and the test suite along with it, necessitating appropriate updates to Goblins in the process. In addition, we found and fixed a few bugs thanks to the new suite.

As of today, both implementations pass the test suite!

Where next?

While we've gotten a lot done during the last 12 months, we still have a ways to go on the road to having an OCapN standard. The higher level work of solidifying a shared set of definitions for OCapN will be done as a community as the OCapN group refines the documents until we reach a foundational consensus.

Once we've agreed upon the first version of the specification documents, with the test suite and Goblins implementations following in lockstep, we plan to bring these documents to a standards body such as IETF or W3C.

We also hope these documents and tools will empower more people to develop OCapN libraries, tools, and implementations, and grow a rich ecosystem so that more secure, distributed applications and games can be easily developed which put the users freedom and privacy first.

Acknowledgments

This work was funded through the NGI Assure Fund, a fund established by NLnet with financial support from the European Commission's Next Generation Internet program. We want to say a big thanks for supporting this work and enabling us to do this important work.